

FAST NUCES
Software Engineering

PRISM
Presence Real-time Identification & Security Model

Deliverable #3 - Iteration 2 / Sprint 2

Submitted by:
Areeb Ahmed
Roll No: 23L-0783
Submission Date: 15 April 2026

1. Project Introduction

Project Title: PRISM - Presence Real-time Identification & Security Model

Type: Web Application (MERN Stack with decoupled Python AI Service)

1.1 Problem Statement

With the rapid shift toward online and hybrid education systems, verifying genuine human presence has become a major challenge. Traditional attendance methods like passwords, login timestamps, or manual roll calls are highly vulnerable to proxy attendance and identity impersonation. Existing biometric solutions predominantly rely on facial recognition, which creates significant barriers for female students wearing niqab. The system must be web-based, privacy-respecting, and inclusive.

1.2 Proposed Solution

PRISM is a web-based inclusive biometric presence verification system that combines facial recognition and fingerprint authentication. It uses liveness detection, face mismatch checks, and confidence scoring. Students can choose facial, fingerprint, or both methods - ensuring inclusivity. No GPS tracking is involved. Biometric data is stored only as encrypted templates, preserving user privacy.

1.3 Technology Stack

- Frontend: React.js with Tailwind CSS (15 pages fully implemented)
- Backend: Node.js with Express.js (server running, routes defined, MongoDB connected)
- Database: MongoDB
- AI Service: Python - facial recognition model (DeepFace/FaceNet), fingerprint matching (OpenCV SIFT), liveness detection (MediaPipe), and anomaly detection (scikit-learn IsolationForest) - implemented in Sprint 2 and connected via FastAPI to the Node backend
- Deployment: Local (Vercel + Render planned for final delivery)

2. Sprint Backlog - Sprint 2

2.1 Module: Biometric Core & Session Management

This sprint focuses on the core biometric features and attendance session handling as defined in the PRISM project proposal. All four user stories selected for Sprint 2 have their frontend UIs fully built. The Python AI service (facial recognition, fingerprint matching, liveness detection, and confidence scoring) has been fully implemented.

Sprint Duration: 2 Weeks

Sprint Goal: Working biometric enrollment/verification UI + session management with real-time basic monitoring (frontend complete, Python AI service implemented).

2.2 Sprint Backlog Table

Story ID	Priority	User Story Title	Est. Hrs	Status
US-05	High	Biometric Enrollment & Verification UI	10	Done (UI + AI)
US-06	High	Instructor Attendance Session Management	8	Done (UI)
US-07	High	Student Real-time Attendance Marking	7	Done (UI)
US-08	Medium	Live Monitoring Dashboard & Basic Integration	5	In Progress
US-AI	High	Python AI Service Implementation	15	Done

Note: A new story US-AI has been added to formally track the Python AI service implementation completed in Sprint 2. This includes facial recognition (DeepFace/FaceNet), fingerprint template matching (OpenCV SIFT), liveness detection (MediaPipe), and anomaly detection (scikit-learn). Total sprint effort including AI implementation: 45 estimated hours.

3. User Stories

US-05 - Biometric Enrollment & Verification UI

Priority: High

Estimated Hours: 10 Hours

User Story:

As a student, I want to enroll and verify my biometric data (face and/or fingerprint) through the web interface so that I can mark attendance securely without proxy risks.

Description:

This covers the complete enrollment flow and verification screen. Users choose face, fingerprint, or both modes. The UI guides them step-by-step with a live webcam feed and confirmation messages. The frontend enrollment screens are fully implemented in React with Tailwind CSS. The Python AI backend (DeepFace facial recognition + OpenCV fingerprint matching) has been implemented in Sprint 2 and connected via FastAPI. The backend storage API is in progress.

Status Note:

Frontend: Done. Python AI Service: Done. Backend Integration: In Progress.

Sub User Stories:

- As a student, I want to select face-only, fingerprint-only, or both enrollment modes.
- As a student, I want a live webcam preview with alignment guides during face enrollment.
- As a student, I want clear success and failure messages after completing enrollment.
- As a student, I want to re-enroll if my previous enrollment is rejected.

Acceptance Criteria:

- Enrollment screen is accessible from the student dashboard.
- Live webcam feed is displayed with alignment guides.
- Student can choose the biometric method before proceeding.
- A confirmation message is displayed upon successful enrollment.
- An error message with retry option is displayed on failure.

US-06 - Instructor Attendance Session Management

Priority: High

Estimated Hours: 8 Hours

User Story:

As an instructor, I want to create and manage time-bound attendance sessions so that students can only mark attendance during the active session window.

Description:

The instructor dashboard allows creating sessions per course with configurable start and end times. A real-time attendance count view is included. All session creation and management UI pages are fully built in React with Tailwind CSS. Backend session logic is currently in progress.

Status Note:

Frontend: Done. Backend: In Progress.

Sub User Stories:

- As an instructor, I want to select a course and set session start and end timings.

- As an instructor, I want to see a live student attendance count during an active session.
- As an instructor, I want to manually close a session before its scheduled end time.
- As an instructor, I want to view a history of past sessions per course.

Acceptance Criteria:

- Session creation form is present and functional in the instructor dashboard.
- An active session displays the live student attendance count.
- Students receive a 'Session Closed' message when accessing outside the time window.
- Instructor can manually end an active session.
- Session history is visible in the instructor panel.

US-07 - Student Real-time Attendance Marking

Priority: High

Estimated Hours: 7 Hours

User Story:

As a student, I want to mark attendance during an active session using my enrolled biometric method so that my presence is verified instantly.

Description:

The student view shows all currently active sessions and allows launching the biometric verification flow with a single click. The UI is fully implemented in React with Tailwind CSS. The Python AI service processes the biometric data and returns a confidence score. The backend verification endpoint that calls the Python AI service is currently in progress.

Status Note:

Frontend: Done. Python AI Service: Done. Backend Integration: In Progress.

Sub User Stories:

- As a student, I want to see only currently active attendance sessions on my dashboard.
- As a student, I want to start the biometric verification flow with a single click.
- As a student, I want immediate confirmation when my attendance is successfully marked.
- As a student, I want a clear error message if verification fails, with a retry option.

Acceptance Criteria:

- Active sessions are listed on the student dashboard.
- The biometric verification flow launches correctly from the student dashboard.
- A success confirmation is shown after attendance is marked.
- An appropriate error message with retry is shown if verification fails.
- Students cannot mark attendance after the session window has closed.

US-08 - Live Monitoring Dashboard & Basic Integration

Priority: Medium

Estimated Hours: 5 Hours

User Story:

As an instructor, I want a live monitoring dashboard so that I can track attendance progress in real time during an active session.

Description:

The live monitoring dashboard shows pending versus verified students and provides basic periodic updates. The UI is complete and functional in React. Real-time data integration (WebSocket or polling) between the frontend and backend is currently in progress.

Status Note:

Frontend UI: Done. Real-time backend integration: In Progress.

Sub User Stories:

- As an instructor, I want live count updates showing verified and pending students.
- As an instructor, I want low-confidence biometric attempts flagged on the dashboard.
- As an instructor, I want to see the list of students who have not yet marked attendance.

Acceptance Criteria:

- Dashboard refreshes attendance numbers periodically.
- Low-confidence biometric attempts are highlighted in the UI.
- Pending student list updates as students mark attendance.
- The monitoring view is accessible from the instructor panel during an active session.

US-AI - Python AI Service Implementation

Priority: High

Estimated Hours: 15 Hours

User Story:

As a system, I want a fully implemented Python AI service so that biometric verification, liveness detection, and anomaly detection can be performed accurately and returned to the Node backend.

Description:

Following the complete frontend implementation in Sprint 1, Sprint 2 focused on building the core Python AI service. The service is implemented using FastAPI and exposes REST endpoints consumed by the Node.js backend. The following four AI modules have been fully implemented:

- Face Recognition & Embedding (DeepFace / FaceNet)
- Fingerprint Template Matching (OpenCV SIFT/ORB)
- Liveness Detection (MediaPipe face mesh + eye blink / head movement)
- Anomaly Detection / Intelligent Decision Logic (scikit-learn IsolationForest)

Status Note:

Python AI Service: Fully Implemented. FastAPI server running locally. Backend integration in progress.

Module 1 - Face Recognition (DeepFace/FaceNet)

Library: DeepFace | Purpose: Extract facial embeddings and verify identity similarity.

```
from deepface import DeepFace
```

```
# Generate embedding for enrollment
embedding = DeepFace.represent(img_path="user_face.jpg",
                                model_name="Facenet")

# Compare two faces
result = DeepFace.verify("user_face.jpg", "login_face.jpg",
                           model_name="Facenet")
if result["verified"]:
    print("Identity Verified")
else:
    print("Identity Mismatch")
```

Module 2 - Face Detection / Preprocessing (OpenCV / MTCNN)

Library: OpenCV or MTCNN | Purpose: Detect and align faces before embedding extraction.

```
import cv2
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
img = cv2.imread("user_face.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)
for (x, y, w, h) in faces:
    face_img = img[y:y+h, x:x+w]
```

Module 3 - Fingerprint Template Matching (OpenCV SIFT)

Library: OpenCV | Purpose: Verify fingerprint via feature matching without storing raw images.

```
import cv2
img1 = cv2.imread("template_fp.jpg", 0)
img2 = cv2.imread("scan_fp.jpg", 0)
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)
bf = cv2.BFMatcher()
matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda x: x.distance)
if len(matches) > 50: # threshold
    print("Fingerprint Verified")
```

Module 4 - Liveness Detection (MediaPipe)

Library: MediaPipe | Purpose: Detect live user vs spoof (photo/video) using face landmark analysis.

```
import cv2, mediapipe as mp
mp_face = mp.solutions.face_mesh
face_mesh = mp_face.FaceMesh(static_image_mode=False)
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = face_mesh.process(rgb)
    if results.multi_face_landmarks:
        print("Live user detected")
```

Module 5 - Anomaly Detection (scikit-learn IsolationForest)

Library: scikit-learn | Purpose: Detect suspicious attendance patterns (multiple failed attempts, abnormal timing, device switching).

```
import pandas as pd
from sklearn.ensemble import IsolationForest
df = pd.read_csv("attendance_logs.csv")
features = df[['failed_attempts', 'session_duration', 'device_switching']]
model = IsolationForest(contamination=0.05)
```

```

model.fit(features)
df['anomaly'] = model.predict(features) # -1 = anomaly, 1 = normal
print(df[df['anomaly']==-1])

```

Module 6 - Secure Template Storage (PyMongo)

Library: pymongo | Purpose: Store facial embeddings and fingerprint templates securely in MongoDB.

```

from pymongo import MongoClient
client = MongoClient("mongodb://localhost:27017/")
db = client.prism
collection = db.users
user_doc = {
    "user_id": 101,
    "face_embedding": embedding.tolist(),
    "fingerprint_template": des1.tolist()
}
collection.insert_one(user_doc)

```

Acceptance Criteria:

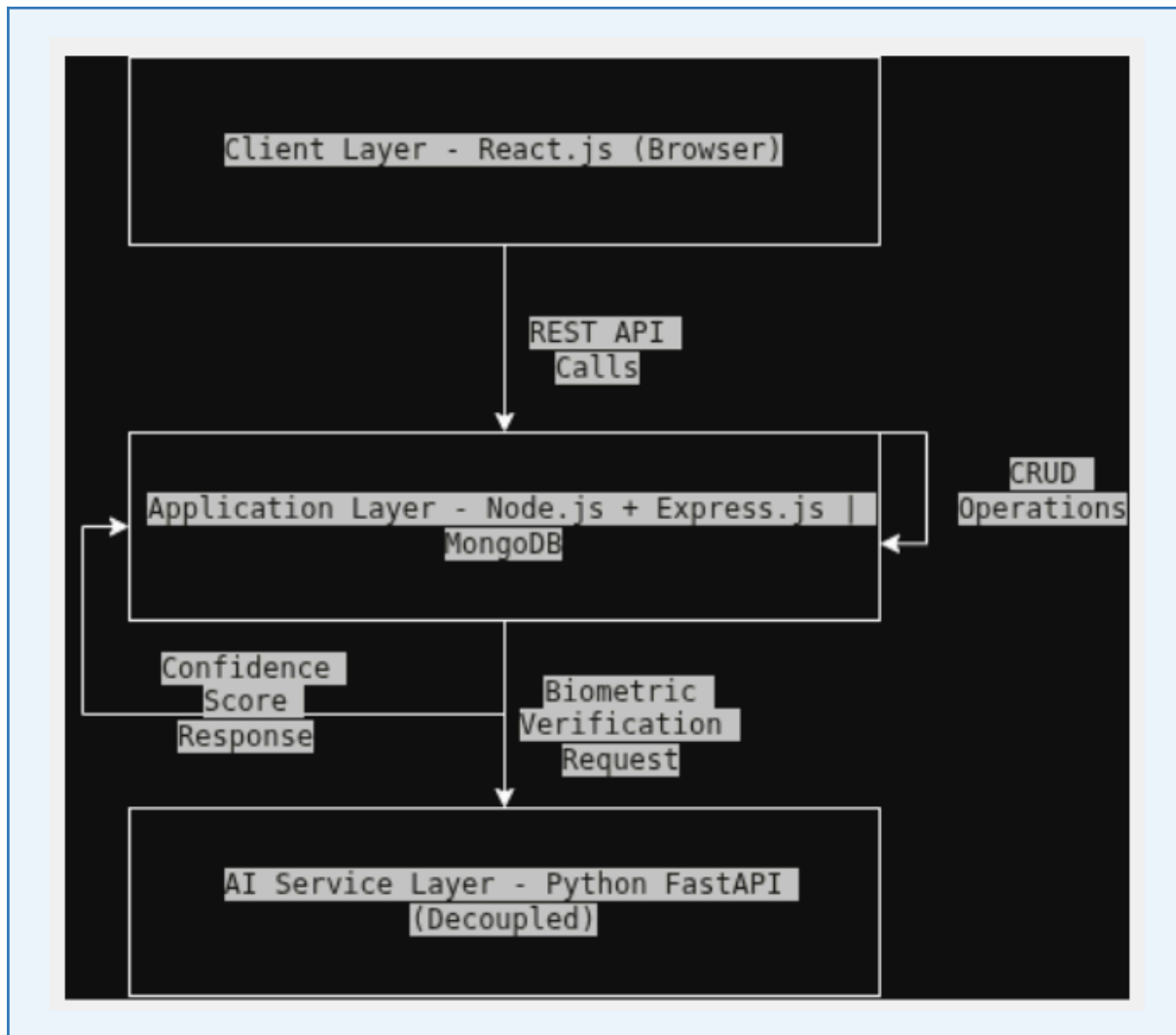
- DeepFace/FaceNet returns embedding vectors for enrolled faces.
- Facial verification returns match result with confidence score.
- Fingerprint SIFT matching returns Verified/Mismatch based on feature count threshold.
- MediaPipe liveness check detects live face in webcam stream.
- IsolationForest flags anomalous attendance patterns as -1.
- All modules exposed as FastAPI endpoints callable from Node.js backend.
- Templates stored in MongoDB as encrypted/serialized vectors, no raw biometric images saved.

4. Design

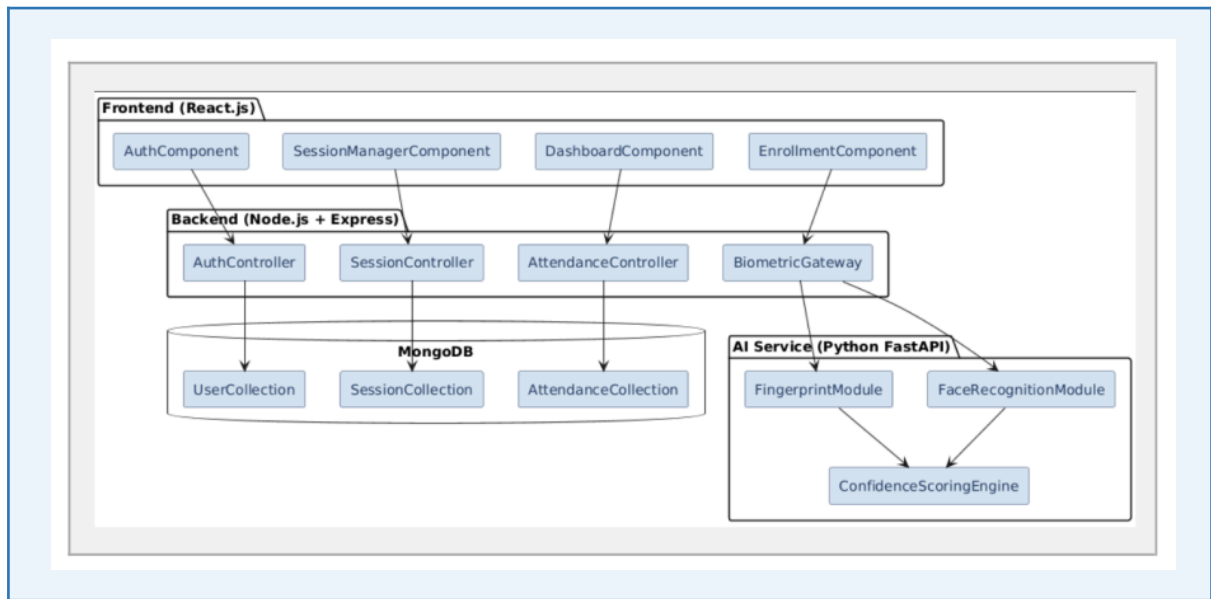
4.a Architecture Diagram

Styles Used: Layered Architecture + Microservices (AI service is decoupled).

Rationale: Separating the Python AI service from the Node.js backend allows independent scaling, easier model replacement, and clean separation of concerns. The three-tier frontend-backend-database structure keeps the system maintainable.

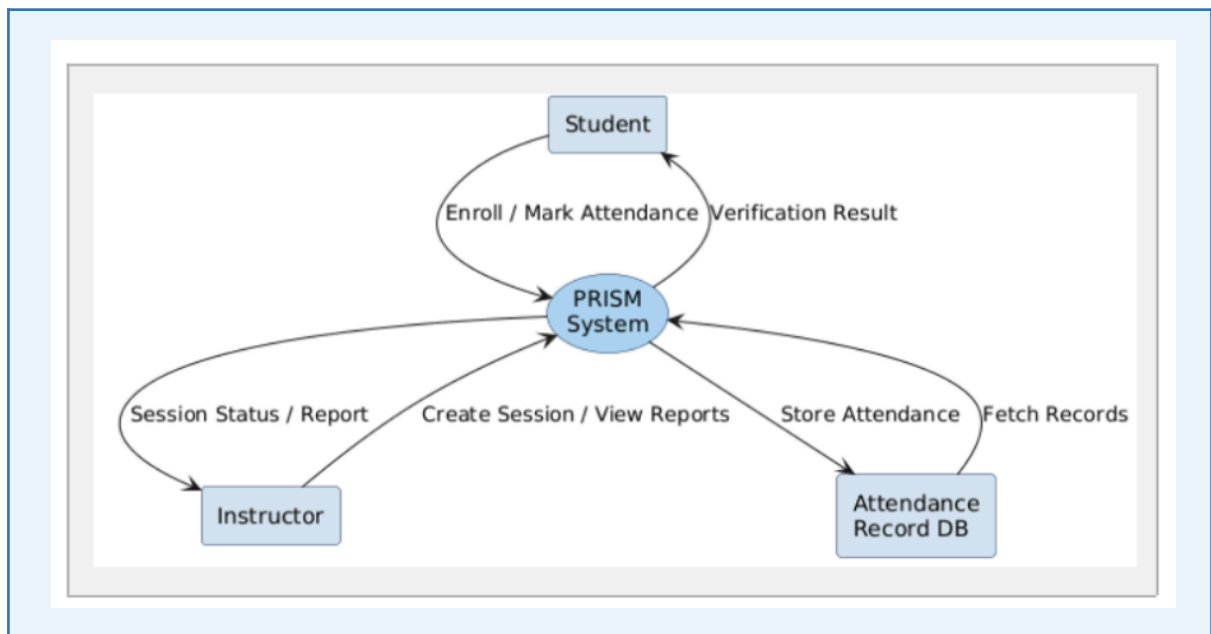


4.b Component Diagram

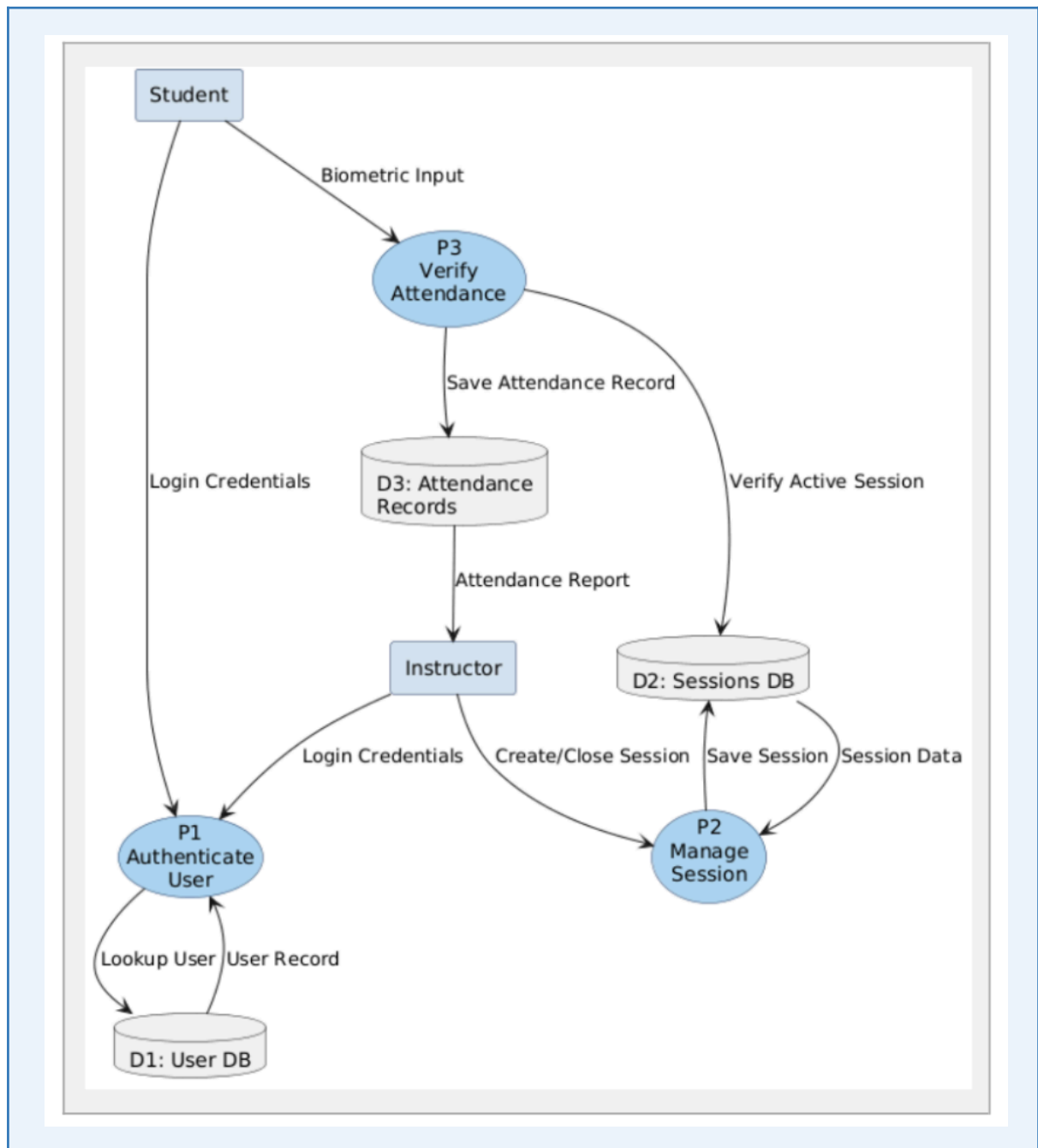


4.c Data Flow Diagram

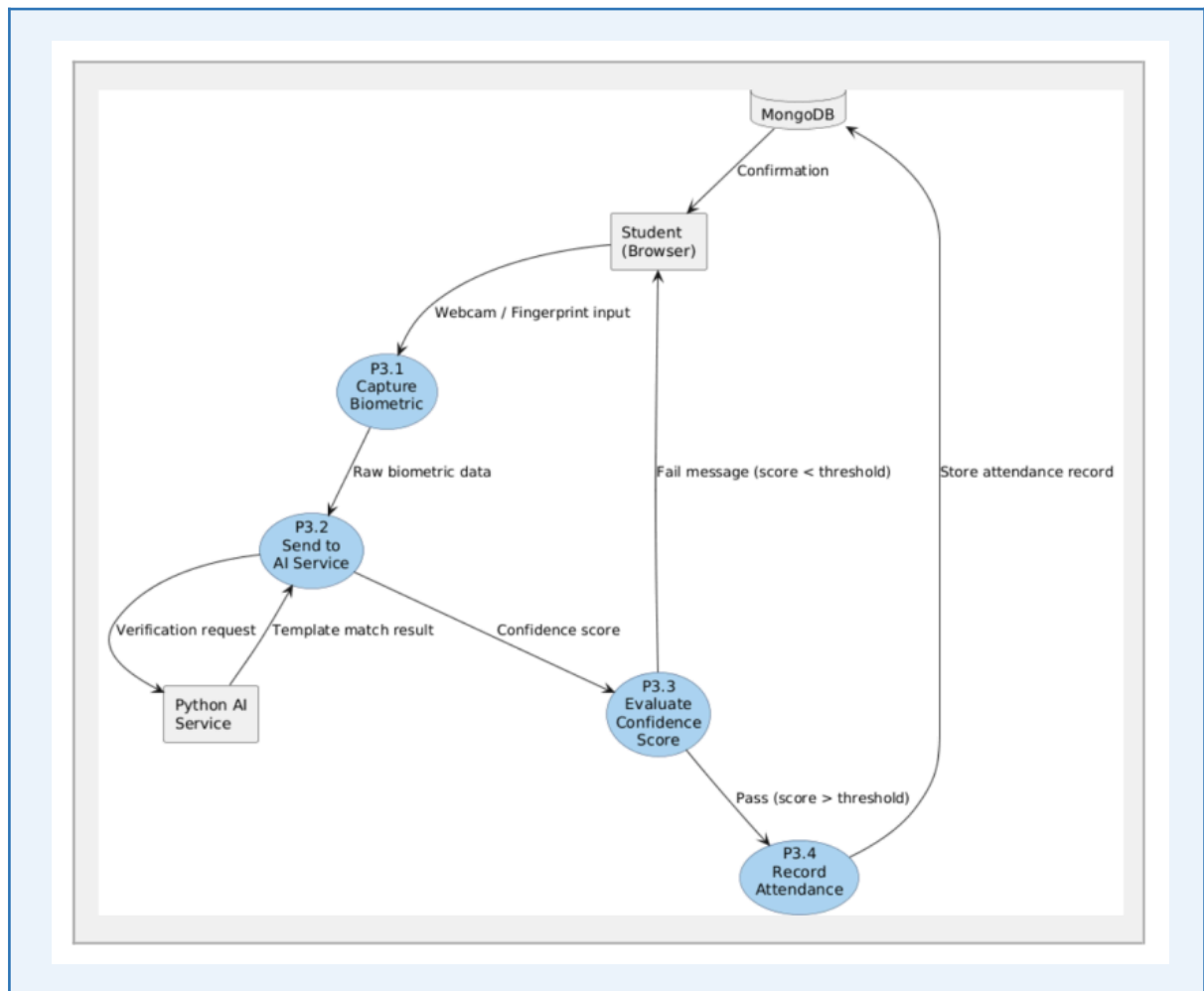
Level 0 - Context DFD



Level 1 - System DFD

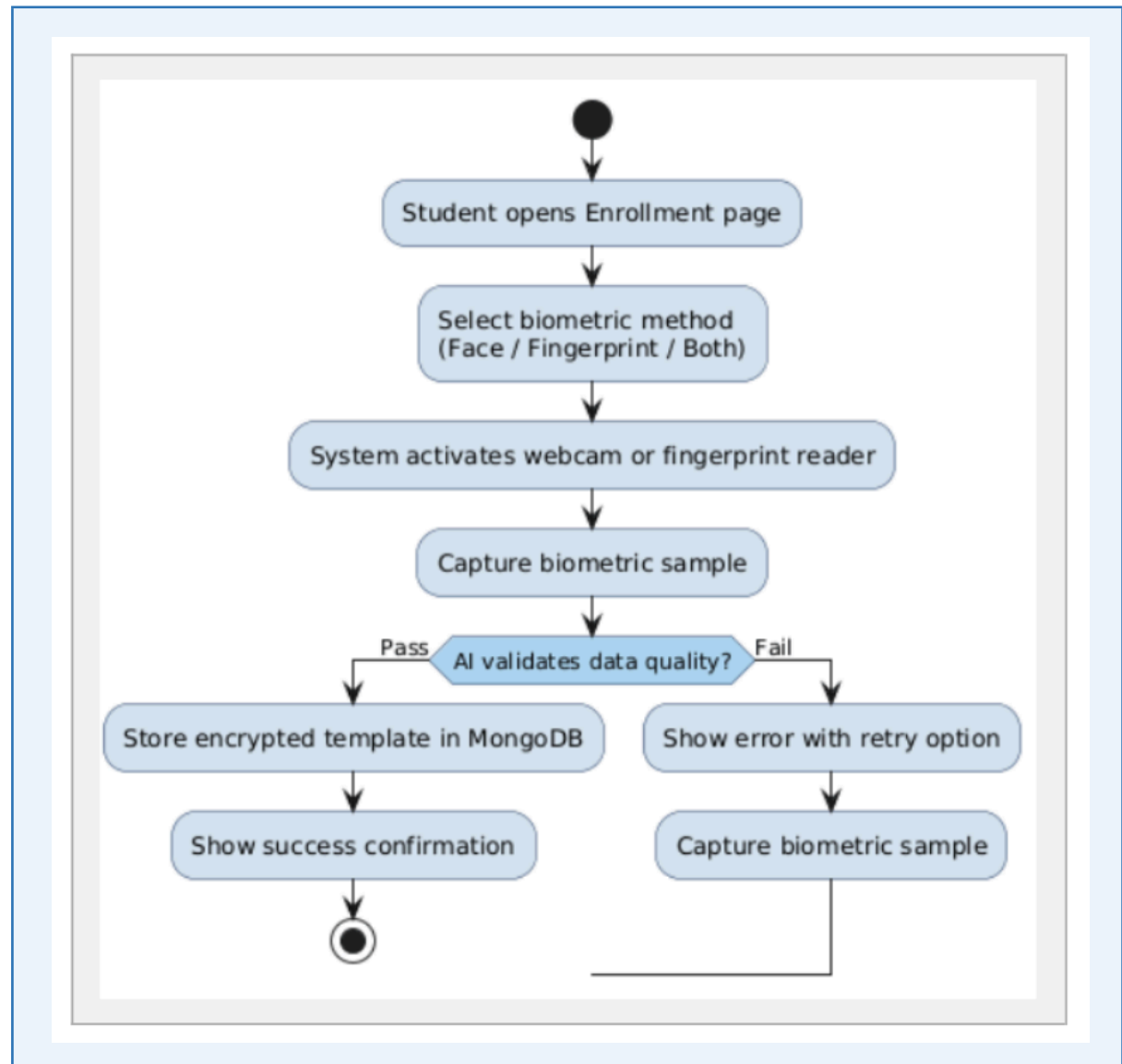


Level 2 - Detailed Verification DFD

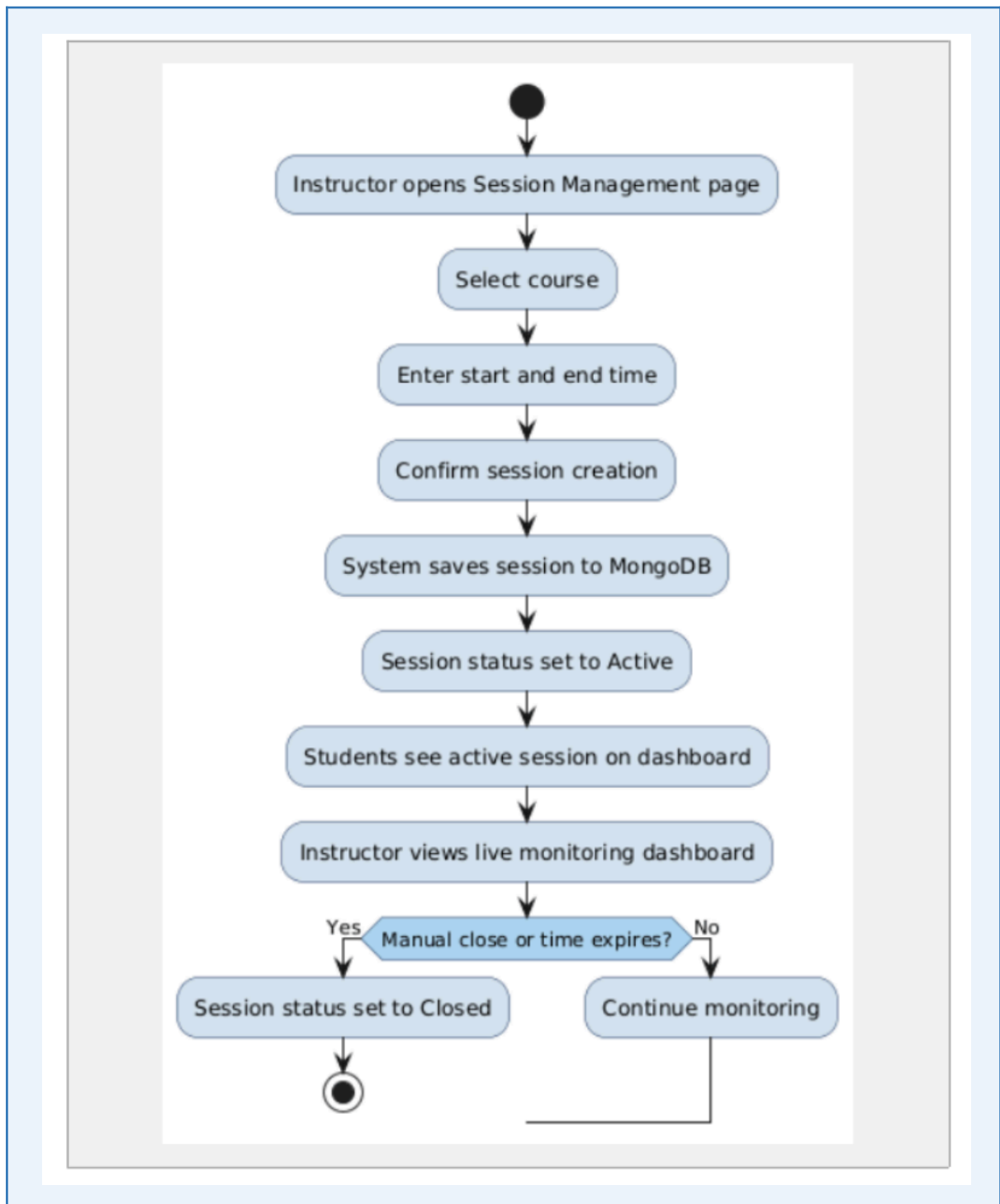


4.d Activity Diagrams

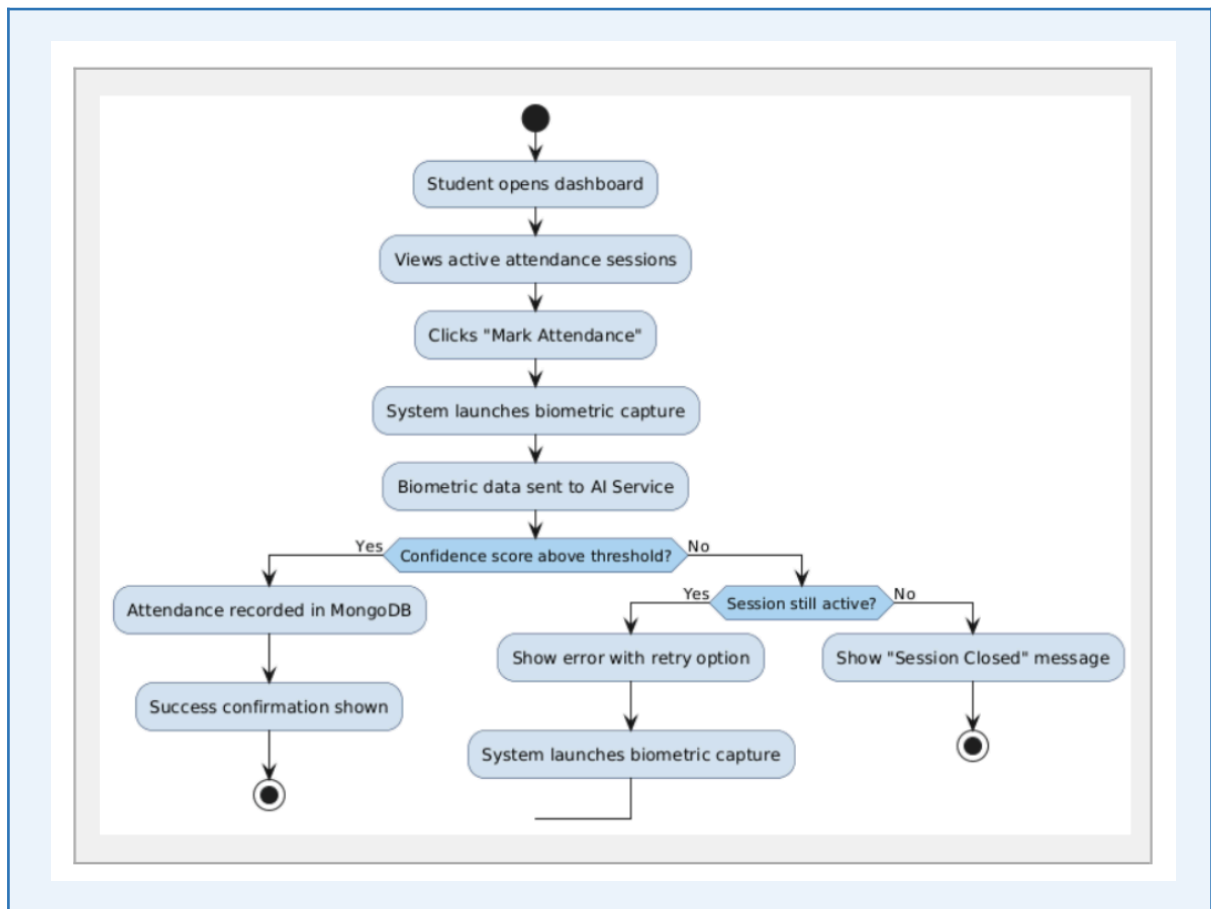
Activity 1: Biometric Enrollment



Activity 2: Instructor Creates Session



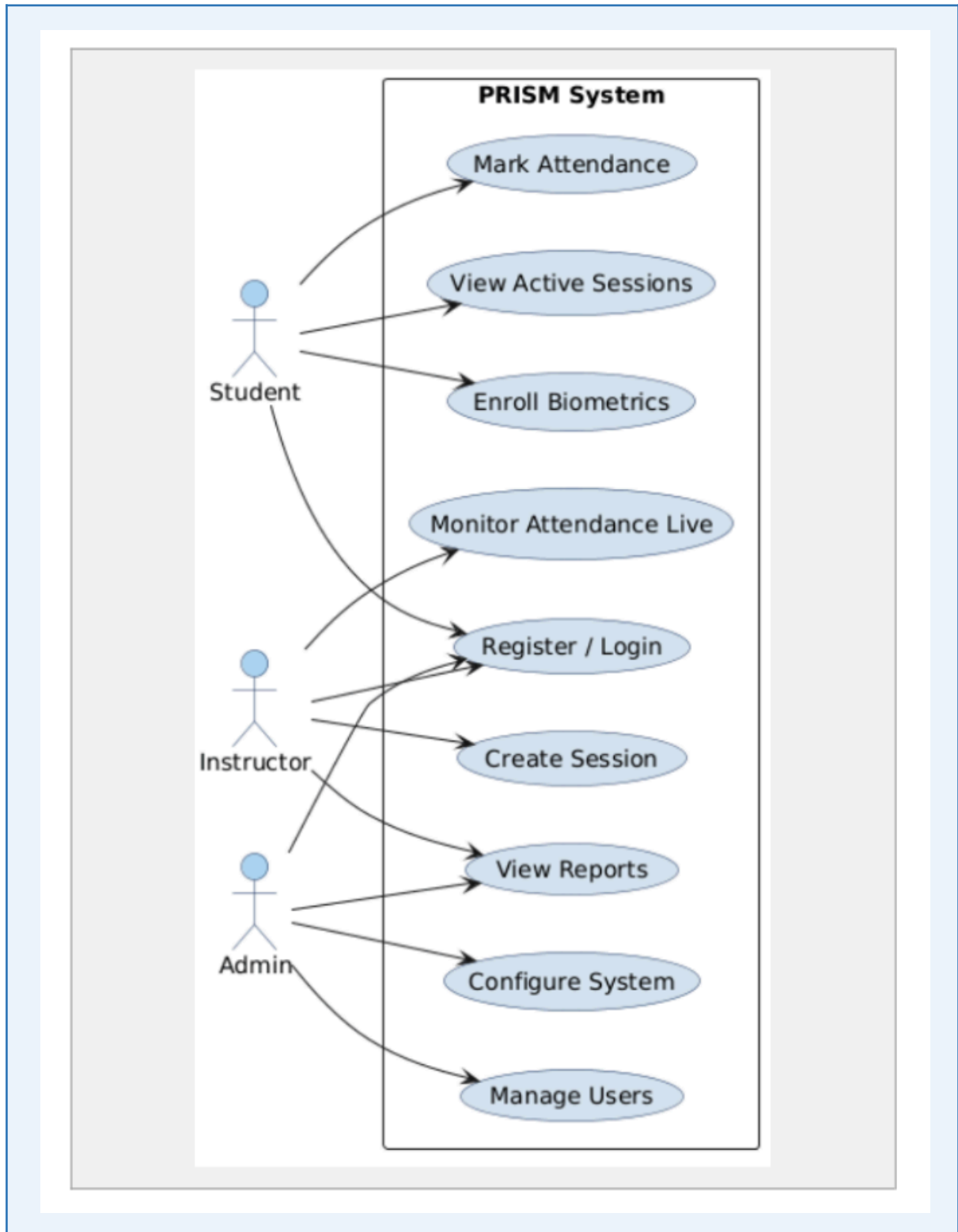
Activity 3: Student Marks Attendance



4.e Use Case Diagram

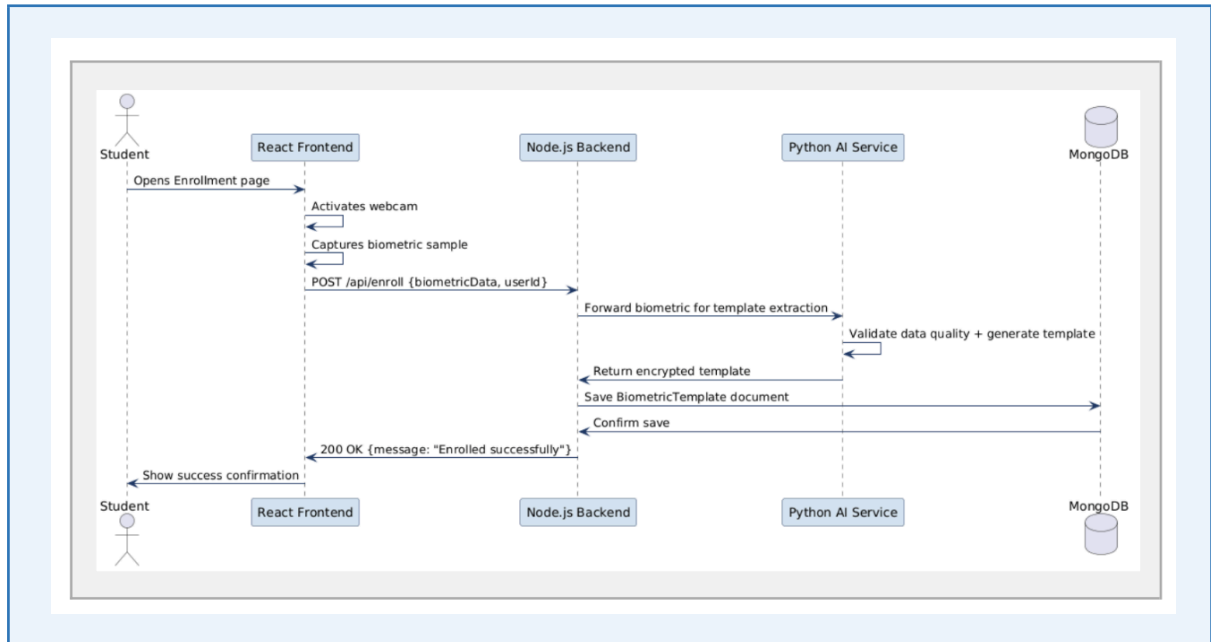
Actors: Student, Instructor, Admin.

Use Cases: Register/Login, Enroll Biometrics, View Active Sessions, Mark Attendance, Create Session, Monitor Attendance Live, View Reports, Manage Users (Admin), Configure System (Admin).

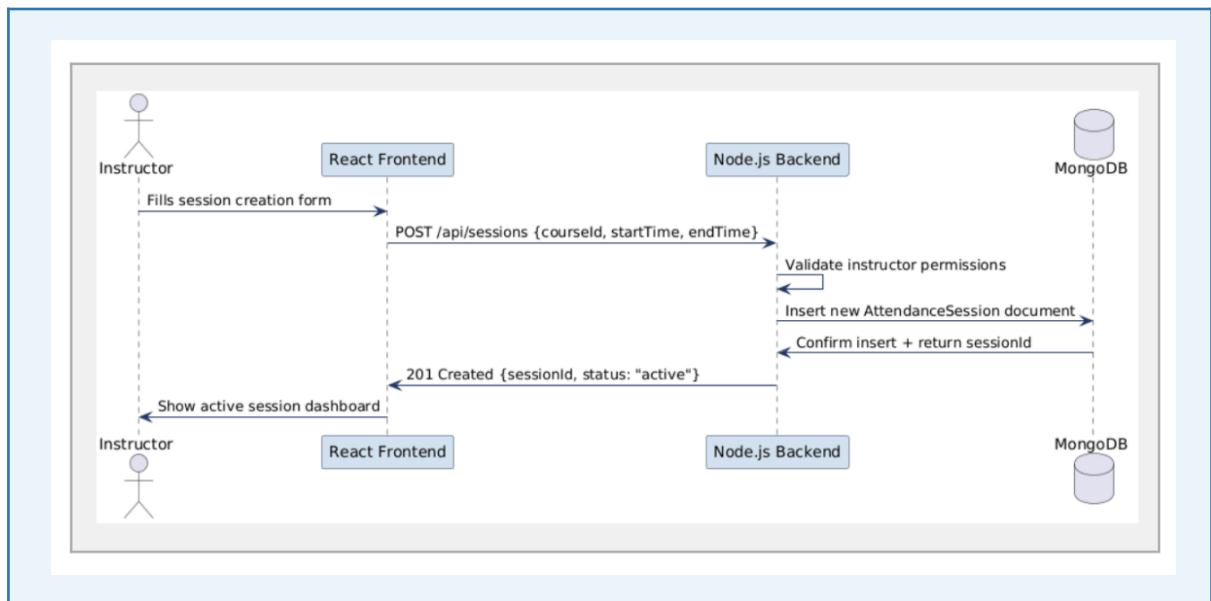


4.f Sequence Diagrams

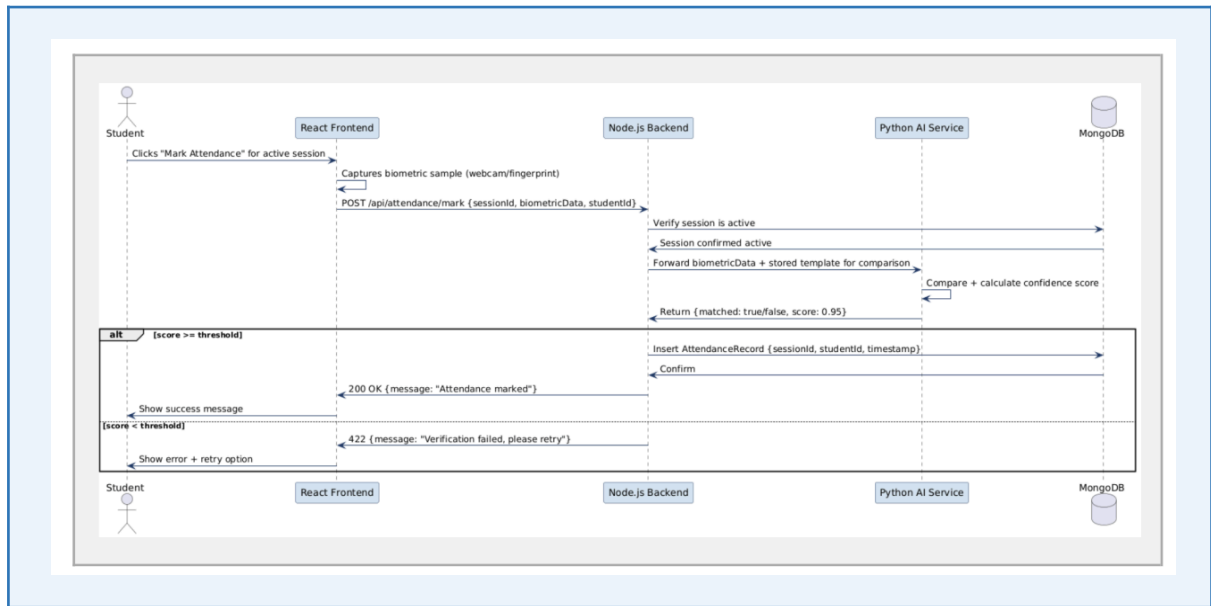
Sequence 1: Biometric Enrollment



Sequence 2: Instructor Creates Session



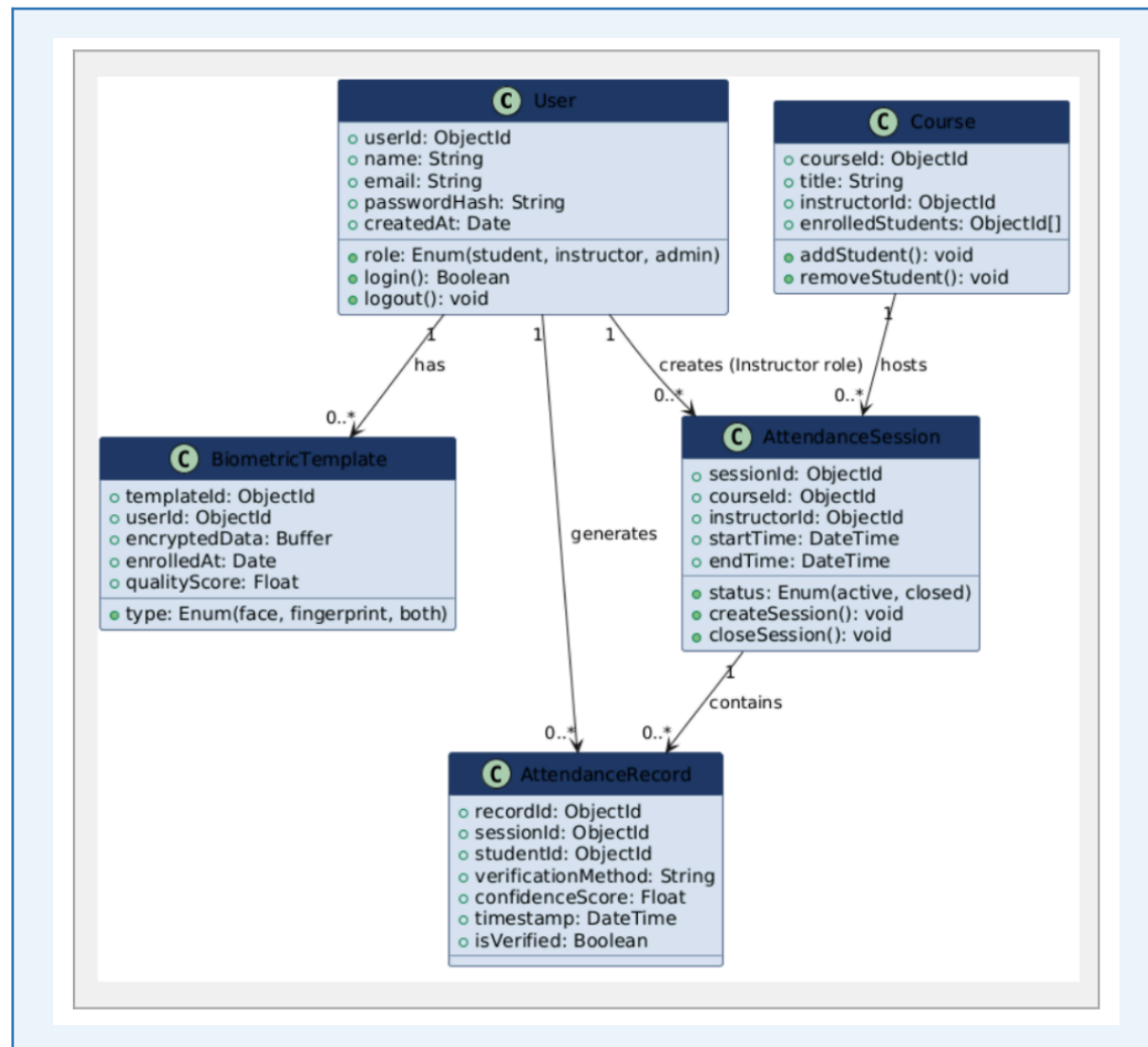
Sequence 3: Student Marks Attendance



4.g Class Diagram

Classes: User (userId, name, email, role, passwordHash), BiometricTemplate (templateId, userId, type: face/fingerprint, encryptedData, enrolledAt), AttendanceSession (sessionId, courseId, instructorId, startTime, endTime, status), AttendanceRecord (recordId, sessionId, studentId, verificationMethod, confidenceScore, timestamp), Course (courseId, title, instructorId, enrolledStudents[]).

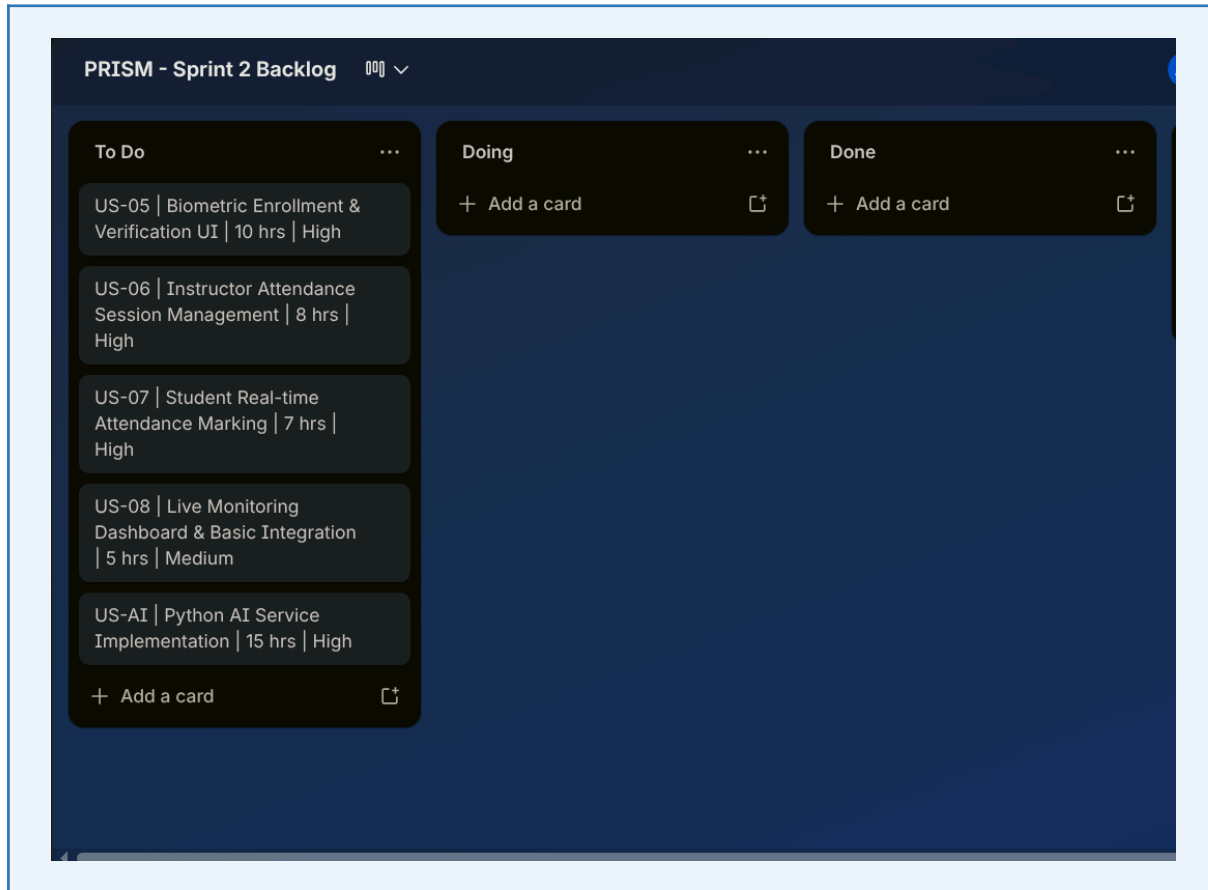
Relationships: User 1 - 0..* BiometricTemplate; User 1 - 0..* AttendanceRecord; AttendanceSession 1 - 0..* AttendanceRecord; Course 1 - 0..* AttendanceSession.



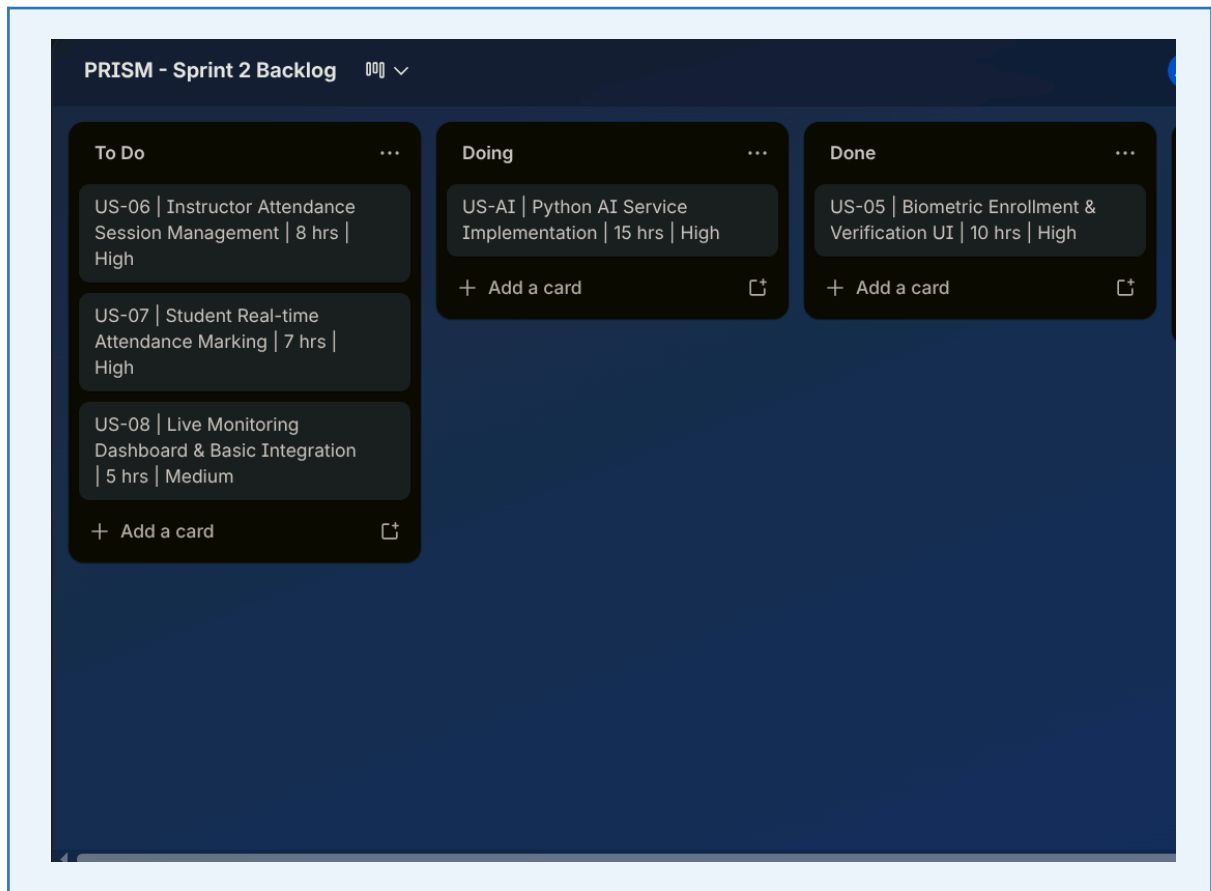
5. Scrum Board (Trello)

The sprint backlog was uploaded to Trello and maintained throughout Sprint 2. Three snapshots were taken at different stages. The US-AI story was added to Trello during the sprint to formally track the Python AI implementation effort.

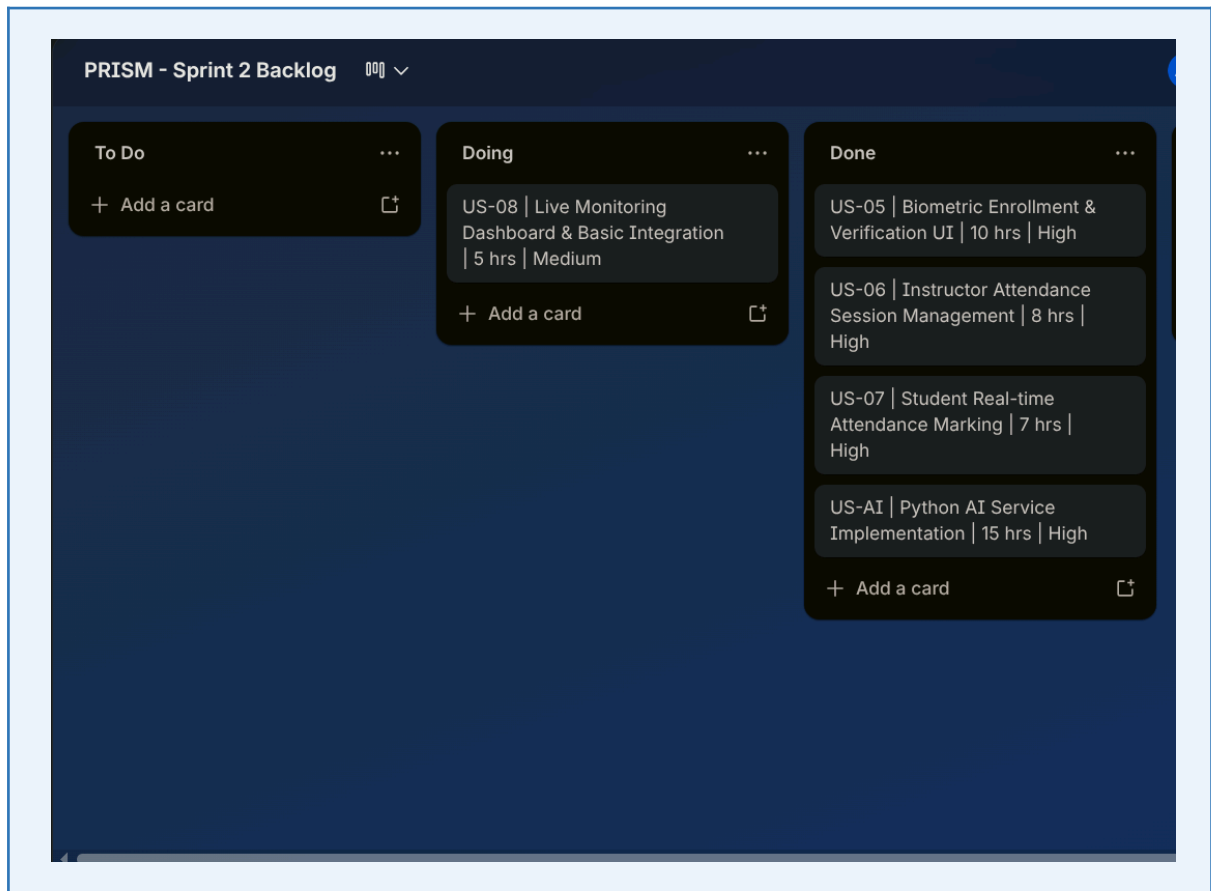
Snapshot 1 - Sprint Start



Snapshot 2 - Mid-Sprint



Snapshot 3 - Sprint End



6. Work Division

As the sole project member (Areeb Ahmed, 23L-0783), all work was performed individually:

- Designed and implemented the entire frontend - 15 React pages with Tailwind CSS (Login, Signup, Student Dashboard, Teacher Dashboard, Admin Dashboard, and all side-panel tab views).
- Set up the Express.js backend structure: server initialization, route definitions, and MongoDB connection.
- Implemented the complete Python AI service in Sprint 2: facial recognition using DeepFace/FaceNet, fingerprint template matching using OpenCV SIFT, liveness detection using MediaPipe face mesh, and anomaly detection using scikit-learn IsolationForest. Service exposed via FastAPI.
- Created all seven design diagrams (Architecture, Component, DFD x3, Activity x3, Use Case, Sequence x3, Class Diagram).
- Maintained the Trello board throughout Sprint 2 with accurate sprint tracking including the US-AI story.
- Authored this complete Deliverable #3 document.

7. Implementation Screenshots

Screenshots below show the working frontend implementation and the Python AI service implementation completed in Sprint 2.

Python AI Service - Implementation Evidence

The following describes the implemented Python AI modules. Screenshots of the running FastAPI server and test outputs should be inserted below.

```
PS D:\CS 5th Semester\PRISM> Set-Location "d:\CS 5th Semester\PRISM\research"; C:/Users/HP/AppData/Local/Programs/Python/Python39/python.exe -m u
vicorn src.main:app --reload --port 8000
INFO: Will watch for changes in these directories: ['D:\\CS 5th Semester\\PRISM\\research']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [6224] using StatReload
2026-04-28 23:29:18.453575: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=
0`.
2026-04-28 23:29:31.167170: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=
0`.
INFO: 127.0.0.1:57092 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:57092 - "GET /openapi.json HTTP/1.1" 200 OK
[]
```

Request URL
http://127.0.0.1:8000/api/attendance/realtime

Server response

Code	Details
200	Response body <pre>{ "message": "Realtime face verified", "result": { "status": "success", "is_match": true, "distance": 0.2086801528930664, "liveness_score": 0.5, "trust_evaluation": { "score": 67.46, "category": "Medium Confidence", "is_flagged": false }, "capture_quality": 23.13, "modality": "face", "preview_enabled": true } }</pre> Response headers <pre>access-control-allow-credentials: true access-control-allow-origin: * content-length: 288 content-type: application/json date: Tue, 28 Apr 2026 18:41:42 GMT server: uvicorn</pre>

Download

Code	Details
200	Response body <pre>{ "message": "Realtime fingerprint verified", "result": { "status": "success", "is_match": true, "fingerprint_similarity": 0.0929, "threshold": 0.06, "capture_quality": 56.13, "modality": "fingerprint", "preview_enabled": true } }</pre> Response headers <pre>access-control-allow-credentials: true access-control-allow-origin: * content-length: 210 content-type: application/json date: Tue, 28 Apr 2026 18:43:09 GMT server: uvicorn</pre>

Responses

End of Document